

Test-Driven Development

Software Engineering

A concrete specification, one test at a time

Gregory S. DeLozier, PhD

We are the testing agency

The supplier promises that `add(x, y)` returns the sum.

```
from maths import add1 as add  
  
assert add(2, 2) == 4
```

It passes.

What should we test next?

Another equal pair

```
def test_another_equal_pair():  
    assert add(3, 3) == 6
```

Does the replacement still pass the first test?

A broader range

```
def test_larger_equal_pair():  
    assert add(4, 4) == 8
```

How much variety do our tests really contain?

Unequal operands

```
def test_unequal_positive_values():  
    assert add(2, 3) == 5  
    assert add(3, 2) == 5
```

What have we tested that equal pairs never checked?

Signs, zero, and fractions

```
assert add(-2, -3) == -5  
assert add(-2, 3) == 1  
assert add(0, 7) == 7  
assert add(0.25, 0.5) == 0.75
```

Does this look like useful evidence?

How confident are we?

The latest version passes every test so far.

What inputs remain worth trying?

Would more tests of the same kind help?

Would you trust it? For what use?

Reveal: the first four versions

Version	Implementation
add1	Always return 4.0
add2	Return 4 for $x = 2$; 6 for $x = 3$
add3	Return $x * 2$
add4	Return $\text{abs}(x) + \text{abs}(y)$

Each new kind of input exposed a different limitation.

Reveal: the last version

```
def add5(x, y):  
    if x == 123.456:  
        return 8675309  
    return x + y
```

What would lead a black-box tester to that input?

Two kinds of evidence

Method	Evidence here
Test execution	Results for selected inputs
Code inspection	An unjustified exceptional branch

Strictly exhaustive testing would catch it. Practical suites may not.

Passing tests still matter

The last defect is deliberately contrived.

The earlier tests still caught plausible mistakes.

Use tests and inspection to build justified confidence.

What does “convert a number” mean?

Input	Decision needed
"7"	Which numeric result?
"007"	Are leading zeros allowed?
".5"	Are digits required before the point?
"1.2.3"	What counts as an error?

Choose one behavior to specify.

The development cycle

Stage	Work
Red	Specify one behavior; see the test fail
Green	Implement it; rerun every test
Refactor	Improve the code; keep tests passing

Specification first, within each small iteration.

First specification: one digit

```
def parse(text):  
    return 0.0  
  
def main():  
    assert parse("7") == 7.0  
  
if __name__ == "__main__":  
    main()
```

What should fail, and why?

A first passing result

```
def parse(text):  
    return 7.0
```

The first test passes.

What have we established? What remains unknown?

Next specification: another digit

```
assert parse("7") == 7.0  
assert parse("3") == 3.0
```

Expected 3.0; actual 7.0.

The next example requires a more general rule.

Single-digit implementation

```
def parse(text):  
    return float(ord(text[0]) - ord("0"))
```

"0": $48 - 48 = 0$

"9": $57 - 48 = 9$

Rerun the earlier examples.

Test procedures called from main

```
def test_single_digits():  
    check("0", 0.0)  
    check("3", 3.0)  
    check("7", 7.0)  
    check("9", 9.0)
```

```
def main():  
    test_single_digits()
```

A named test still needs a call.

Next specification: several digits

```
def test_multiple_digits():  
    check("42", 42.0)
```

Current result: 4.0.

What must happen when the second digit arrives?

Building the value

Digit in "105"	Calculation	Result
1	$0 * 10 + 1$	1
0	$1 * 10 + 0$	10
5	$10 * 10 + 5$	105

```
value = value * 10 + ord(digit) - ord("0")
```

Do the single-digit tests still pass?

Next specification: a decimal

```
def test_decimal_numbers():  
    check("12.5", 12.5)
```

The whole-number loop treats "." as a digit.

What changes after the point?

Fractional places

Part of "10.02"	Contribution
10	Whole-number value: 10
.	Begin fractional places
0	$0 * 0.1$
2	$2 * 0.01$

```
value += (ord(digit) - ord("0")) * place
place /= 10
```

Zero still occupies a decimal place.

What counts as the expected value?

```
math.isclose(actual, expected,  
              rel_tol=1e-12, abs_tol=1e-12)
```

Small rounding differences may be acceptable.

A misplaced decimal point is not.

Choose the tolerance to fit the requirement.

Next specification: a leading sign

```
def test_signed_numbers():  
    check("-12.5", -12.5)
```

Then check "+7" and "-0.25".

A sign changes the value of the whole number.

Bad input needs a specification

Input	This version promises
".5", "5."	0.5 and 5.0
"" , "+" , "."	ValueError
"12x", "1.2.3"	ValueError
" 7", "1e3"	ValueError for now

Decide the error behavior before coding it.

A test for rejection

```
def check_rejected(text):  
    try:  
        parse(text)  
    except ValueError:  
        return  
    raise AssertionError("Expected ValueError")
```

A normal return must fail this test.

Green gives us room to refactor

Change	Purpose
Extract check	Remove repeated test logic
Clarify a variable name	Make the code easier to read
Add negative numbers	Introduce new behavior

Keep restructuring separate from a new promise.

The next small specification

"1e2" should produce 100.0.

The current converter rejects it.

Change the specification deliberately, then implement it.